



Having FUN with gcc, the C Compiler

In this article we will discuss a few important options from among the hundreds provided by the gcc compiler to serve the needs of different types of programmers – student as well as professional.

First of all, we need to distinguish between GCC and gcc. GCC (GNU Compiler Collection) is a compiler system that provides compilers for the following programming languages – C, C++, Objective-C, FORTRAN, Ada, Go and D. The C compiler provided by GCC is called gcc. As the title suggests, this article is about gcc – the C compiler, and not about GCC, the compiler system. There's also one question that needs to be answered before we proceed any further, and that is, "Do I really believe that the use of a compiler can ever be considered fun?" The answer is, yes. Exploring the features of gcc is not only informative but it could also be fun.

Let us begin by discussing two of the most commonly used options of gcc. Consider the simple C program *pgm1.c* given below. This and all the other programs along with their associated files discussed in this article can be downloaded from opensourceforu.com/article_source_code/Nov19funwithgcc.zip.

```
#include<stdio.h>
#define NUMBER 5
int main()
{
    int a = NUMBER;
    printf("Square of %d is %d\n\n", a,
a*a);
    return 0;
}
```

The gcc command *gcc pgm1.c* will compile the program *pgm1.c* to produce the Linux-executable file with the default name *a.out*. The executable file *a.out* can be executed with the command *./a.out*. The executable file produced by gcc can be given a specific name by using the option *-o*. The command *gcc pgm1.c -o exfile* will create an executable file called *exfile*. This executable file can be executed with the command *./exfile*. Figure 1 shows the execution of the program *pgm1.c*.

The compilation process of a program

The compilation of a C program involves a number of steps like preprocessing,

generating assembly code, object code, executable code, etc. If required, gcc allows us to do this process step by step till we get an executable file. This is especially useful to computer science teachers because such an explanation will give students a clear understanding of the compilation process.

First, from the C file *pgm1.c*, let us create the preprocessed code in the file *pgm1.i* with the command *gcc -E pgm1.c -o pgm1.i*. Next, we will use the preprocessed code in *pgm1.i* to obtain the assembly code in the file *pgm1.s* with the command *gcc -S pgm1.s*. Next, the assembly code in the file *pgm1.s* is used to obtain the object code (relocatable machine code) in file *pgm1.o* with the command *gcc -c pgm1.s*. Finally, the object code in file *pgm1.o* is used to produce the executable (absolute machine code) file *exfile* with the command *gcc pgm1.o -o exfile*.

Figure 2 shows this step by step compilation process of the C program *pgm1.c*. In the figure, we can see that the macro *NUMBER* defined by the line of code *#define NUMBER 5* and used in the line of code *int a = NUMBER;* in